UML

Application ANBG PAS / PAO / PTA / Actions

- Version: 1.1
- Date: 2026-03-09

Ce document fournit des diagrammes UML en syntaxe Mermaid.

1. Diagramme De Cas D Utilisation

```
flowchart LR
    Admin[admin]
    DG[dg]
    Planif[planification]
    Dir[direction]
    Serv[service]
    Agent[agent]
    Cab[cabinet]

    subgraph UseCases
        UC1[Gerer PAS]
        UC2[Gerer PAO]
        UC3[Gerer PTA]
        UC4[Creer et assigner action]
        UC5[Renseigner suivi periodique]
        UC6[Soumettre cloture action]
        UC7[Valider action chef]
        UC8[Valider action direction]
        UC9[Consulter dashboard/pilotage]
        UC10[Consulter reporting et exporter]
        UC11[Consulter alertes]
        UC12[Administrer referentiels]
        UC13[Consulter audit]
        UC14[Lire notifications]
    end

    Admin --> UC1
    Admin --> UC2
    Admin --> UC3
    Admin --> UC4
    Admin --> UC9
    Admin --> UC10
    Admin --> UC11
    Admin --> UC12
    Admin --> UC13
    Admin --> UC14

    DG --> UC1
    DG --> UC2
```

```
    DG --> UC3
    DG --> UC8
    DG --> UC9
    DG --> UC10
    DG --> UC11
    DG --> UC14

    Planif --> UC1
    Planif --> UC2
    Planif --> UC3
    Planif --> UC4
    Planif --> UC9
    Planif --> UC10
    Planif --> UC11
    Planif --> UC14

    Dir --> UC2
    Dir --> UC3
    Dir --> UC4
    Dir --> UC8
    Dir --> UC9
    Dir --> UC10
    Dir --> UC11
    Dir --> UC14

    Serv --> UC3
    Serv --> UC4
    Serv --> UC7
    Serv --> UC9
    Serv --> UC10
    Serv --> UC11
    Serv --> UC14

    Agent --> UC5
    Agent --> UC6
    Agent --> UC14

    Cab --> UC9
    Cab --> UC10
    Cab --> UC11
    Cab --> UC13
    Cab --> UC14
```

2. Diagramme De Classes (Metier)

```
classDiagram
    class User {
      +id
      +name
      +email
```

```
    +role
    +direction_id
    +service_id
    +is_agent
  }

  class Direction {
    +id
    +code
    +libelle
  }

  class Service {
    +id
    +direction_id
    +code
    +libelle
  }

  class Pas {
    +id
    +titre
    +periode_debut
    +periode_fin
    +statut
  }

  class PasAxe {
    +id
    +pas_id
    +direction_id
    +code
    +libelle
  }

  class PasObjectif {
    +id
    +pas_axe_id
    +code
    +libelle
  }

  class Pao {
    +id
    +pas_id
    +direction_id
    +annee
    +statut
  }

  class Pta {
```

```
    +id
    +pao_id
    +direction_id
    +service_id
    +statut
  }

  class Action {
    +id
    +pta_id
    +responsable_id
    +type_cible
    +frequence_execution
    +statut_dynamique
    +statut_validation
  }

  class ActionWeek {
    +id
    +action_id
    +numero_semaine
    +est_renseignee
    +progression_reelle
    +progression_theorique
  }

  class ActionKpi {
    +id
    +action_id
    +kpi_global
    +statut_calcule
  }

  class Justificatif {
    +id
    +justifiable_type
    +justifiable_id
    +action_week_id
    +categorie
  }

  class ActionLog {
    +id
    +action_id
    +niveau
    +type_evenement
    +cible_role
  }

  class Notification {
    +id(uuid)
```

```
      +notifiable_type
      +notifiable_id
      +data(json)
      +read_at
    }

    Direction "1" --> "many" Service
    Direction "1" --> "many" User
    Service "1" --> "many" User

    Pas "1" --> "many" PasAxe
    PasAxe "1" --> "many" PasObjectif

    Pas "1" --> "many" Pao
    Direction "1" --> "many" Pao
    Pao "1" --> "many" Pta
    Service "1" --> "many" Pta

    Pta "1" --> "many" Action
    User "1" --> "many" Action : responsable
    Action "1" --> "many" ActionWeek
    Action "1" --> "1" ActionKpi
    Action "1" --> "many" ActionLog
    Action "1" --> "many" Justificatif
    ActionWeek "1" --> "many" Justificatif
    User "1" --> "many" Notification
```

3. Diagramme De Sequence

Soumission Et Validation D Une Action

```
sequenceDiagram
    participant A as Agent
    participant UI as UI Actions
    participant S as ActionTrackingService
    participant N as WorkspaceNotificationService
    participant C as Chef Service
    participant D as Direction

    A->>UI: Saisit periode + justificatif
    UI->>S: submitWeek()
    S-->>UI: progression + statut recalcules

    A->>UI: Soumettre cloture action
    UI->>S: submitClosureForReview()
    S->>N: notifyActionSubmittedToChef()
    N-->>C: Notification "Action soumise"

    C->>UI: Evaluer (valider/rejeter)
    UI->>S: reviewClosureByChef()
```

```
    alt validation chef
      S->>N: notifyActionReviewedByChef(approved=true)
      N-->>D: Notification "Action a valider"
    else rejet chef
      S->>N: notifyActionReviewedByChef(approved=false)
      N-->>A: Notification "Action rejetee"
    end

    D->>UI: Evaluer (valider/rejeter)
    UI->>S: reviewClosureByDirection()
    alt validation direction
      S->>N: notifyActionReviewedByDirection(approved=true)
      N-->>A: Notification "Action validee officiellement"
    else rejet direction
      S->>N: notifyActionReviewedByDirection(approved=false)
      N-->>A: Notification "Action rejetee direction"
    end
```

4. Diagramme D Etats

Etats De Validation Action

```
stateDiagram-v2
    [*] --> non_soumise
    non_soumise --> soumise_chef: soumettre cloture
    soumise_chef --> validee_chef: chef valide
    soumise_chef --> rejetee_chef: chef rejette
    rejetee_chef --> soumise_chef: agent corrige et resoumet
    validee_chef --> validee_direction: direction valide
    validee_chef --> rejetee_direction: direction rejette
    rejetee_direction --> soumise_chef: agent corrige et resoumet
    validee_direction --> [*]
```